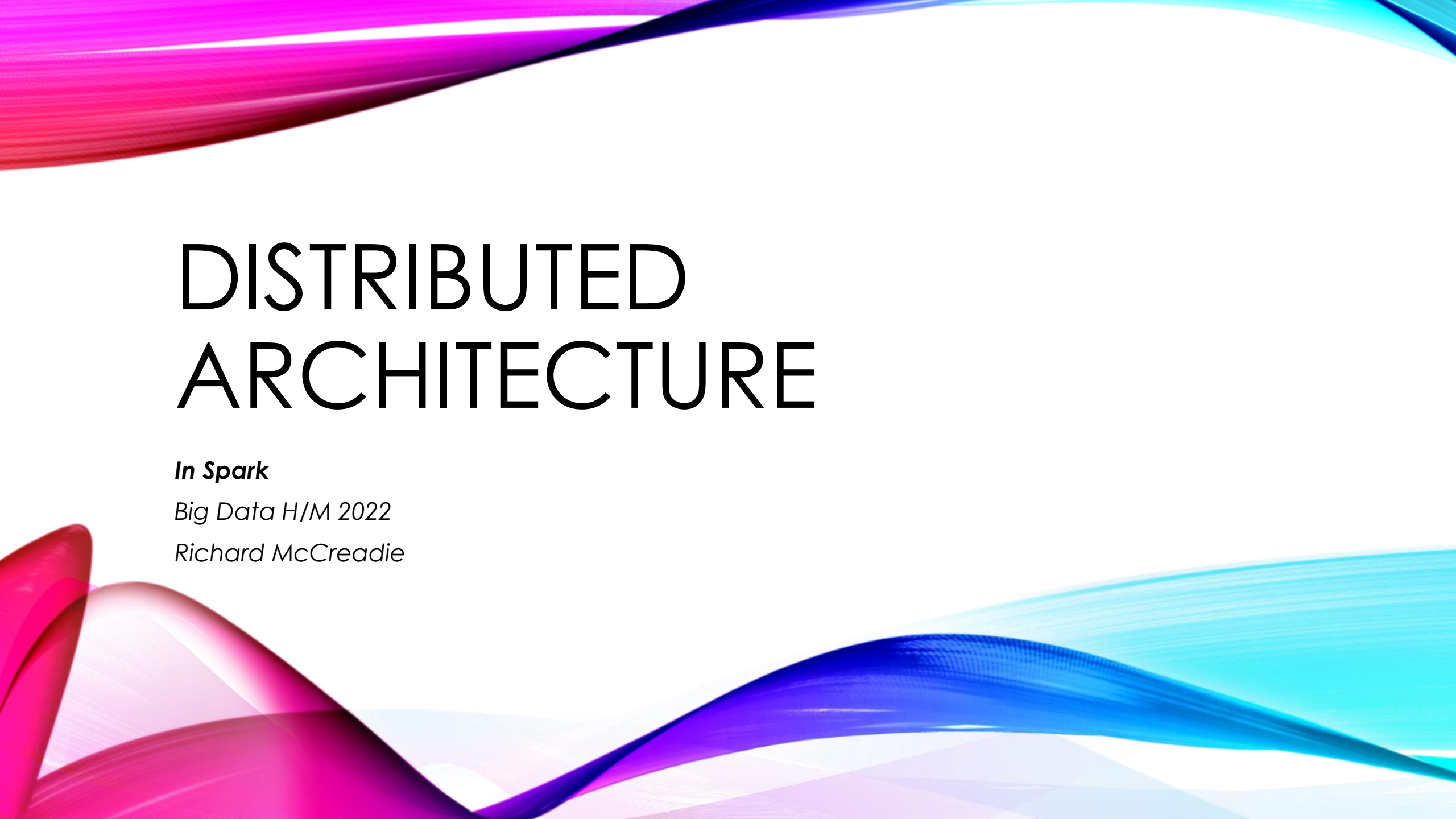




DISTRIBUTED ARCHITECTURE

In Spark

Big Data H/M 2022

Richard McCreadie



DATA...

- Up to this point we have covered the main transformation functions that you will use to convert data from one type to another
 - However, given this high-level foundation covering what you can do with data, we now need to understand a bit more what goes on under-the-hood
- This can be important so you can **reason about your application** and hence **predict how efficient it will be**

FIRST, SOME TERMINOLOGY

- **Job**: A parallel computation consisting of **multiple tasks** that gets spawned in response to a **Spark Action**
 - Your driver program may create multiple jobs, depending on how many actions are contained within it
- **Action**: A spark function that returns some data back to the driver program or persistent storage device
 - Actions create Jobs, which do computation
- **Task**: A single instance of a spark function (e.g. a transformation, like map or reduce)
 - A Job is an ordered grouping of tasks, some of which may be carried out in parallel, while others need to be processed sequentially

EXAMPLE

Job

Task



Task



Action



```
File hadoopDIR = new File("resources/hadoop/");  
System.setProperty("hadoop.home.dir", hadoopDIR.getAbsolutePath());
```

```
SparkConf conf = new SparkConf()  
    .setMaster("local[2]")  
    .setAppName("SparkTutorial1");
```

```
SparkSession spark = SparkSession  
    .builder()  
    .config(conf)  
    .getOrCreate();
```

```
// -----  
// Spark Application Topology Starts Here  
// -----
```

```
//-----  
// Data Input  
//-----  
// Lets read in some statistics of Steam games from a file  
Dataset<Row> steamGamesAsRowTable = spark  
    .read()  
    .option("header", "true")  
    .csv("data/Steam/games_features.sample.csv");
```

```
//-----  
// Data Transformations  
//-----
```

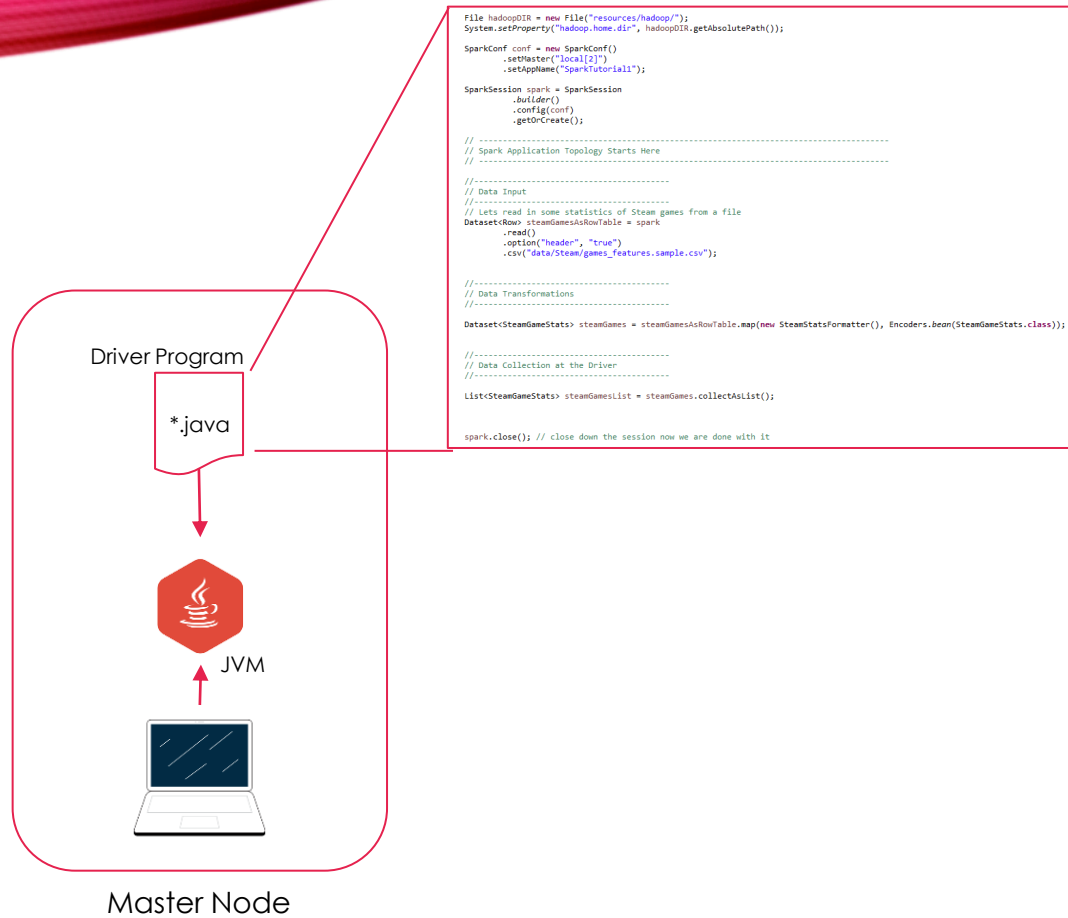
```
Dataset<SteamGameStats> steamGames = steamGamesAsRowTable.map(new SteamStatsFormatter(), Encoders.bean(SteamGameStats.class));
```

```
//-----  
// Data Collection at the Driver  
//-----
```

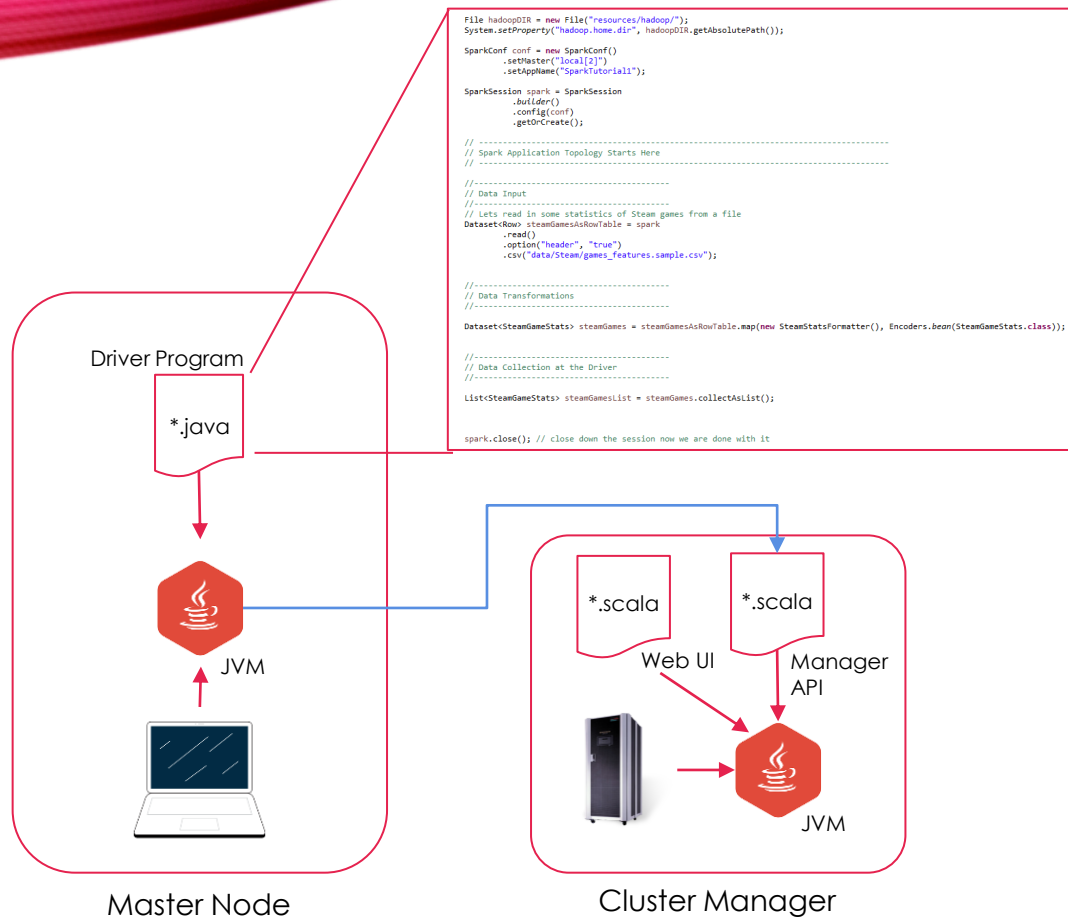
```
List<SteamGameStats> steamGamesList = steamGames.collectAsList();
```

```
spark.close(); // close down the session now we are done with it
```

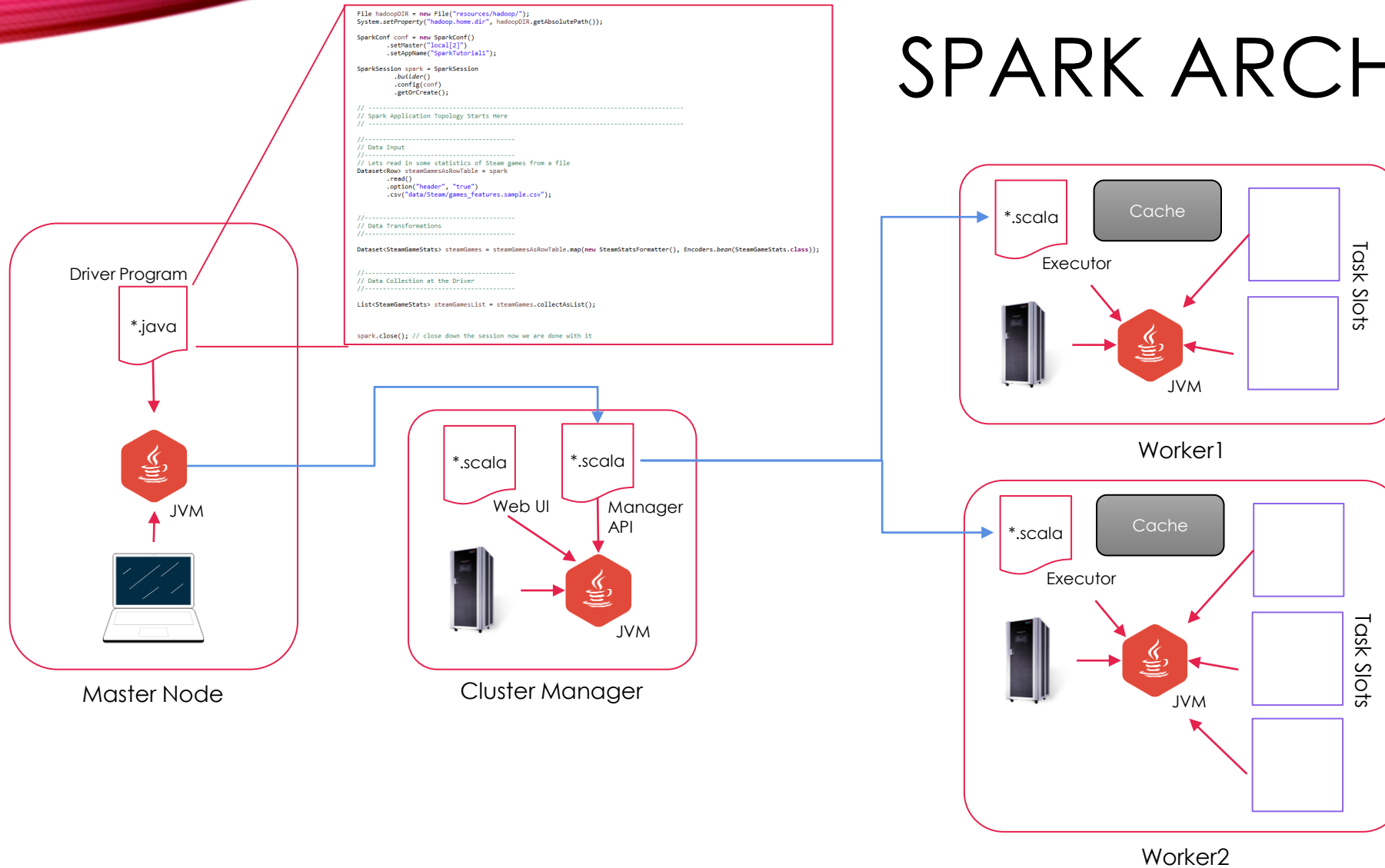
SPARK ARCHITECTURE



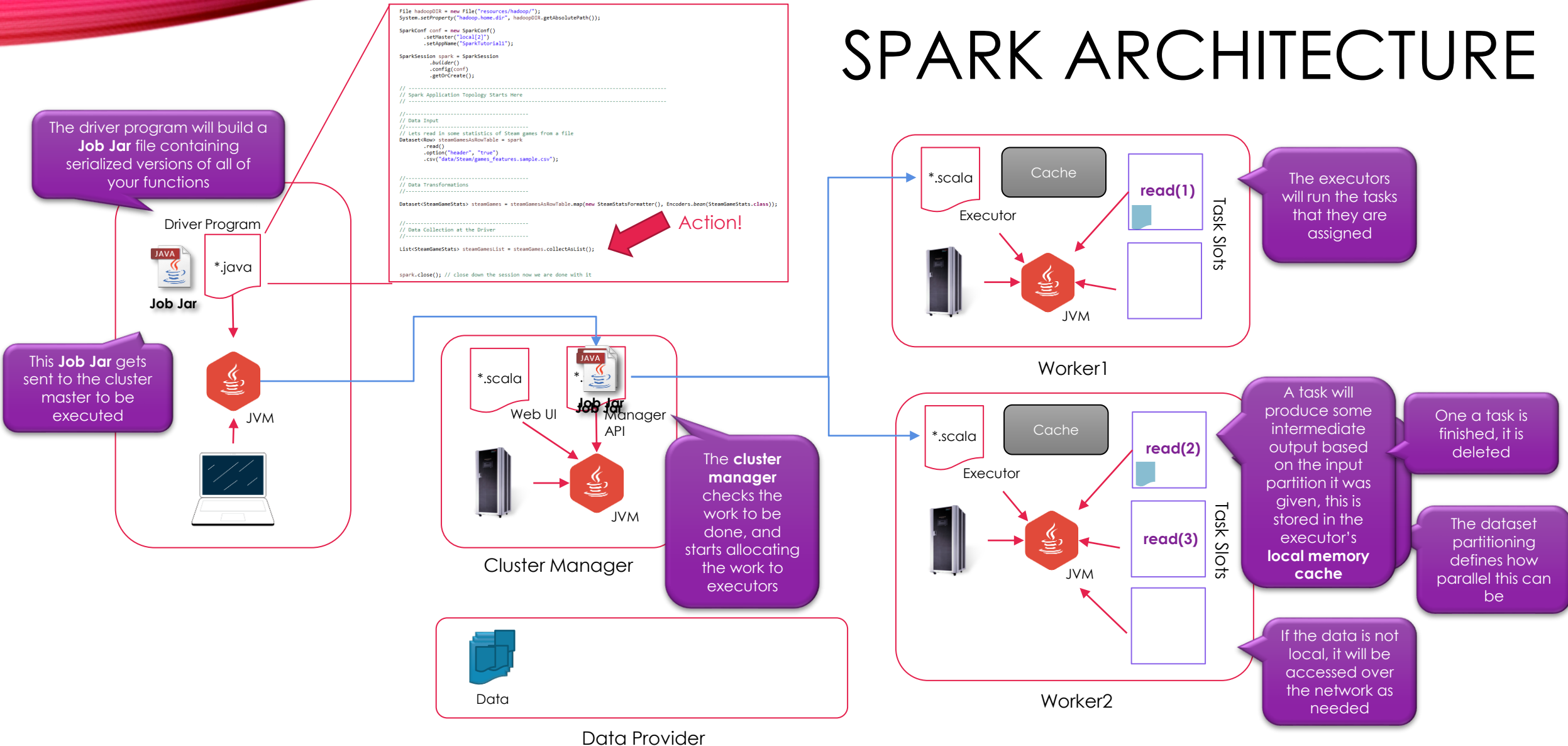
SPARK ARCHITECTURE



SPARK ARCHITECTURE



SPARK ARCHITECTURE



SPARK ARCHITECTURE

